Smart Exam Prep System

Complete Implementation Plan

Mobile App | Smart India Hackathon 2026

Tagline: "Never walk into an exam not knowing what you don't know."

This document outlines the complete, phase-wise implementation plan for building the Smart Exam Prep System as a mobile application — covering architecture, module-wise development steps, sprint timeline, team roles, and deployment strategy for the hackathon.

1. Project Overview

Smart Exam Prep System is a mobile-first platform that automatically detects a student's syllabus gaps — whether caused by absenteeism or in-class inattention — and converts them into a prioritized, adaptive, and verified revision plan before exams. It combines relational data modeling (attendance × lecture plan), lightweight NLP (focus-check verification), and LLM-grounded content generation (explainers and practice questions) into a single closed-loop system.

Core Modules

- Gap Detection Engine (attendance × lecture-plan join)
- Confidence vs. Competence Mismatch Detector
- Forgetting-Curve Based Revision Scheduler
- Auto-Generated Practice Questions & Explainers (LLM)
- Answer-Writing Pattern Analyzer
- "Explain It Back" Verification
- Exam-Day Time-Budget Simulator
- Stress-Aware Adaptive Load
- Cross-Subject Interference Mapper
- "Last 24 Hours" Mode
- Post-Exam Feedback Loop

2. System Architecture

High-level flow:

React Native (Expo) Mobile App → REST / WebSocket → FastAPI Backend → PostgreSQL (Supabase) | LLM API (explainers/questions) + Sentence-Transformers microservice (focus-check NLP) | Firebase Cloud Messaging (push alerts)

Architecture Layers

Layer	Components	Technology
Mobile Frontend	Student App, Teacher Portal, Admin View	React Native (Expo), NativeWind, React Navigation
Backend / API	Gap Detection, Ranking, Scheduling, Notifications	FastAPI (Python)
Database	Attendance, Lecture Plan, Gaps, Quiz Results	PostgreSQL via Supabase
ML / NLP	Focus-check similarity, embeddings	sentence-transformers (MiniLM-L6-v2)
LLM Layer	Explainer & question generation (grounded)	Claude API
Notifications	Same-day absence/gap alerts	Firebase Cloud Messaging
Storage	Peer notes, cheat sheets	Supabase Storage
Offline Cache	Local gap-list & explainer cache	expo-sqlite, AsyncStorage

3. Database Schema (Core Tables)

Table	Key Fields
students	id, name, class, section
teachers	id, name, subjects
lecture_plan	id, date, period, subject, topic, subtopics[], weightage_tag
attendance	student_id, date, period, status
focus_check	student_id, date, period, self_rating, summary_text, flagged
gap_topics	student_id, topic_id, reason, priority_score, status
quiz_results	student_id, topic_id, question_id, correct, confidence_rating
past_papers	subject, topic, marks_weightage, year
notes_repository	topic_id, uploaded_by, file_url, upvotes
exam_feedback	student_id, question_id, correct, mapped_gap_id

4. Phase-Wise Implementation Plan

Phase 1: Planning & Design (Day 1)

- Finalize feature scope for MVP (Gap Detection, Confidence Detector, Scheduler, LLM Explainers)
- Design database schema and entity relationships
- Wireframe key screens: Student Dashboard, Gap List, Explainer Card, Teacher Portal
- Set up GitHub repo, Supabase project, and Expo project skeleton

Phase 2: Backend & Database Setup (Day 1-2)

- Create PostgreSQL schema on Supabase (tables above)
- Set up FastAPI project structure with routers per module
- Implement Auth (Supabase Auth) for student/teacher roles
- Seed sample data: lecture plans, attendance, past papers

Phase 3: Core Gap Detection Engine (Day 2)

- Build attendance × lecture-plan join logic (temporal matching)
- Auto-insert absent-topic records into gap_topics table
- Build focus-check API: capture self-rating/summary after each period
- Integrate sentence-transformers microservice for summary-topic similarity scoring
- Auto-flag low-similarity focus checks as gaps

Phase 4: Priority Ranking & Scheduling (Day 2-3)

- Implement weighted priority scoring: exam weightage + foundational importance + time-decay
- Implement simplified forgetting-curve model using quiz history
- Build endpoint to return a sorted, prioritized gap list per student

Phase 5: LLM-Powered Content Generation (Day 3)

- Integrate Claude API for topic explainer generation, grounded in lecture-plan text (RAG-style prompting)
- Generate practice questions calibrated to past-paper style/difficulty
- Cache generated content in DB to avoid repeated API calls

Phase 6: Confidence-vs-Competence Detector (Day 3)

- Build quiz interface capturing both answer and confidence rating
- Compute mismatch score (high confidence + low accuracy) per topic
- Flag mismatched topics as high-risk on dashboard

Phase 7: Mobile App Development (Day 2-4, parallel track)

- Build Student Dashboard: prioritized gap list, countdown, risk indicators
- Build Explainer & Quiz screens with API integration
- Build Teacher Portal: lecture-plan entry, attendance marking, class gap heatmap

- Integrate offline caching (expo-sqlite/AsyncStorage) for gap list and explainers

Phase 8: Notifications (Day 4)

- Set up Firebase Cloud Messaging with Expo push notification service
- Trigger same-day alert when a student is marked absent on a lecture-plan date
- Trigger reminder alerts for scheduled revision (forgetting-curve based)

Phase 9: Testing & Polish (Day 4-5)

- End-to-end test: mark attendance → gap detected → notification → explainer → quiz → verification
- UI polish, loading states, error handling
- Prepare demo data set covering a realistic pre-exam scenario

Phase 10: Demo & Pitch Preparation (Day 5)

- Record/prepare live demo flow (2-3 minute walkthrough)
- Prepare pitch deck aligned with problem → solution → architecture → novelty → impact
- Rehearse Q&A; on technical depth (joins, NLP, LLM grounding, scoring algorithm)

5. Suggested Timeline (5-Day Hackathon)

Day	Focus	Key Deliverable
Day 1	Planning, schema design, project setup	DB schema live, repos initialized
Day 2	Gap Detection Engine + backend core	Working attendance-to-gap pipeline
Day 3	Ranking, scheduling, LLM integration	Prioritized gap list + AI explainers
Day 4	Mobile app screens + notifications	Functional app with push alerts
Day 5	Testing, polish, demo & pitch prep	Stable demo build + pitch deck

6. Suggested Team Roles

Role	Responsibility
Backend Developer	FastAPI services, DB schema, gap-detection & ranking logic
Mobile Developer	React Native app, screens, offline caching, navigation
ML/NLP Engineer	Sentence-transformer similarity service, forgetting-curve model
AI/LLM Integrator	Prompt design, explainer & question generation, grounding logic
UI/UX Designer	Wireframes, Figma prototypes, design consistency
Presenter / PM	Pitch deck, demo script, coordination, research & references

7. Risks & Mitigation

Risk	Mitigation
LLM API latency during live demo	Pre-cache explainers for demo topics; show live generation only once
Real attendance/lecture data unavailable	Use realistic seeded/mock data for demo
Push notification setup complexity	Use Expo's managed push service to reduce native config effort
Time overrun on non-core features	Follow MVP priority order strictly (Section 4, Phases 3-6 first)

8. MVP Scope for Demo

Given hackathon time constraints, prioritize building (in order):

- 1. Gap Detection Engine — must work end-to-end
- 2. Confidence-vs-Competence Mismatch Detector
- 3. Forgetting-Curve based scheduler (simplified)
- 4. Auto-generated explainers/practice questions via LLM

Mock with seed data rather than building live: notes-matching/upvote system, answer-writing analyzer, stress tracking — present as designed/planned if time is short.

9. Research & References

Tech Stack

- Supabase (DB + Auth + Storage) — supabase.com/docs
- Firebase Cloud Messaging — firebase.google.com
- FastAPI — fastapi.tiangolo.com
- React Native (Expo) — docs.expo.dev
- Sentence-BERT (MiniLM-L6-v2) — sbert.net
- Claude API — docs.claude.com

Conceptual Foundations

- Ebbinghaus Forgetting Curve — basis for spaced-repetition scheduling
- SM-2 / FSRS Spaced Repetition Algorithm — super-memo.com/english/ol/sm2.htm
- Confidence-Accuracy Calibration research — basis for mismatch detector

Existing Solutions Reviewed

- Attendance apps (school ERP systems) — track presence only, no comprehension check
- Note-sharing platforms — passive, not personalized or prioritized
- Generic LMS (Google Classroom, Moodle) — host content, don't detect individual gaps